Shared by Rahil. This is a copy of a chat between Claude and Rahil. Content may include unverified or unsafe content that does not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

Report

I want the best practice to send an encrypted otp and password from my fromtend to backend

2 days ago

Use HTTPS (TLS) and send the OTP and password as plain fields in a POST body. Don't add your own encryption in the frontend. TLS already encrypts everything in transit, and any key or algorithm you ship in JavaScript is visible to an attacker, so custom encryption adds complexity without adding protection.

In transit

- HTTPS only, TLS 1.2+ (ideally 1.3). Add an HSTS header so browsers never fall back to HTTP.
- Send credentials in a **POST body**, never in a URL or query string, because URLs end up in logs, browser history and referrers.
- Don't hash the password on the client as a substitute for server hashing. The hash just becomes the new password.

Password on the backend

- Hash with **Argon2id** (or bcrypt or scrypt). Never store plaintext or use a fast hash like SHA-256 or MD5.
- Compare in constant time. The library's `verify()` handles this.
- Never log request bodies on auth routes.

OTP on the backend

- Generate it with a CSPRNG (`crypto.randomInt`, `secrets`), not `Math.random`.
- Store a **hash** of the OTP, not the OTP itself, with a short expiry (2–5 minutes).
- Make it single-use: delete it after a successful check.
- Limit attempts (for example, 5 per OTP) and rate-limit by user and IP.
- Bind the OTP to the specific user and action it was issued for.

Session and request hygiene

- After login, issue the session as an `HttpOnly; Secure; SameSite` cookie, not a token in localStorage.
- Add CSRF protection if you use cookies.
- Return generic errors like "Invalid credentials" so attackers can't tell which users exist.

Frontend example

```js
await fetch("https://api.example.com/auth/login", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  credentials: "include",
  body: JSON.stringify({ email, password, otp }),
});
```

Extra payload encryption (for example, RSA or JWE on top of TLS) is only worth it in special cases, such as compliance rules or when TLS ends at a proxy you don't trust. Even then it doesn't protect you if the frontend itself is compromised.

If you tell me your backend stack (Node, Spring, Django and so on), I can show the hashing and OTP verification code for it.

2 days ago

> Actually security analyst in the organisation are capturing the request with burpsuit and then flagging it as vulnerable.

2 days ago

This is a very common finding in VAPT reports, and in most cases it's a **false positive**. Burp can show the plaintext only because the analyst installed Burp's CA certificate on their own machine. In effect, they're looking at their own data from the inside. TLS is meant to stop *third parties* on the network from reading traffic, not the user who owns the browser. Any data the browser sends, including "encrypted" data, can be seen in DevTools before it leaves.

How to respond to the finding

- Explain that the interception needed a trusted root CA installed on the client, which a remote attacker can't do without already controlling the device. At that point they could just keylog or read JS memory.
- Show that client-side encryption doesn't help here. The key and the logic sit in the JS bundle, so the analyst could decrypt it the same way, or simply **replay the encrypted blob**, which works just as well as the password.
- Point to standard guidance: OWASP's Authentication and Transport Layer Security cheat sheets require TLS plus server-side hashing, not payload encryption.
- Ask them to rate it as informational, or as accepted risk.

Check what they may actually be flagging. Sometimes the real issue is something else Burp revealed:

- Can the captured request be **replayed** and still work? That's a real bug. Make OTPs single-use and expire them quickly.
- Is there no rate limiting or lockout on password or OTP attempts?
- Is the password or OTP echoed in the response, the logs or the URL?

- Are TLS or HSTS weak, or is plain HTTP allowed?

If they still mandate encryption (common in banking or compliance audits), implement it properly so it at least prevents replay:

1. The server exposes an RSA public key. The private key never leaves the server.
2. The frontend uses the Web Crypto API to create a random AES-GCM key, encrypt `{password, otp, nonce, timestamp}`, and wrap the AES key with RSA-OAEP.
3. The server decrypts, rejects stale timestamps and reused nonces, and then hashes and verifies as normal.

Document it as defence-in-depth or a compliance control, not as a fix, so nobody treats it as a replacement for TLS and hashing.

I can write the Web Crypto frontend code and the matching backend decryption if you tell me your backend stack.

2 days ago

> Actually it is a banking and compliance stuff and as they were insisting on encrypting it in the client side using static keys in javascript which on reassessment they have discovered as they are able to locate the key and have again flagged the issue what should i do now. I use angular and springboot in two different linux servers and angular is deployed using nginx

2 days ago

Created 3 files, shared files

Replace the static symmetric key with **asymmetric (public/private key) encryption**. The analysts found a key that can *decrypt*, and that key was sitting in the JavaScript. With RSA, the browser only ever holds the **public key**, which can encrypt but can't decrypt. The private key stays on your Spring Boot server, so finding the key in the bundle gives an attacker nothing.

How it works (code in the three files)

1. Angular calls `GET /api/auth/handshake` and receives the server's public key and a **one-time nonce** that expires in 2 minutes.
2. Angular creates a random AES-256 key for each request, encrypts `{username, password, otp, nonce}` with AES-GCM, and wraps the AES key with RSA-OAEP. It does this with the browser's built-in Web Crypto API, so no library is needed.
3. Spring Boot decrypts, checks the nonce and **deletes it**. A payload captured in Burp and replayed is rejected, which is usually the analysts' next finding.
4. After that, your normal Argon2/bcrypt password check and OTP check run unchanged.

Setting up the key on the Spring Boot server only

```
keytool -genkeypair -alias login-2026-10 -keyalg RSA -keysize 3072 \
  -validity 365 -storetype PKCS12 -keystore login-crypto.p12 \
  -dname "CN=login-crypto"
```

- Set `login.crypto.keystore`, `login.crypto.keystore-password` and `login.crypto.alias`. Supply the password from an environment variable or a vault, never from git.
- Restrict the file with `chmod 600` so only the app user can read it. For a bank, an HSM or vault is the better home for this key, and the auditors will like it.
- Nothing changes on the nginx/Angular server. It holds no secrets.
- If you run multiple Spring instances, move the nonce store to Redis.

Two details that cause bugs

- Java's `OAEPWithSHA-256AndMGF1Padding` silently uses SHA-1 for MGF1, which doesn't match Web Crypto. The code sets both to SHA-256 explicitly.
- Rotate keys yearly by changing the alias. The `kid` field lets you do this cleanly.

What to tell the analysts at reassessment

- The key in the JS is a public key by design and can't decrypt anything. The private key exists only on the backend server.
- A captured request can't be replayed because each nonce is single-use and expires in 2 minutes.
- TLS, server-side hashing, and OTP expiry and rate limiting are still in place underneath.

2 days ago

I want you to give me the code for the same

2 days ago

Here's the full code, ready to copy.

1. Angular: `secure-login.service.ts`

ts

```ts
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { firstValueFrom } from 'rxjs';

interface HandshakeResponse {
  kid: string;
  publicKey: string; // base64 SPKI
  nonce: string;
}

export interface EncryptedPayload {
  kid: string;
  ek: string;
  iv: string;
  ct: string;
}

@Injectable({ providedIn: 'root' })
export class SecureLoginService {
  constructor(private http: HttpClient) {}

  async login(username: string, password: string, otp: string) {
    const payload = await this.encrypt({ username, password, otp });
    return firstValueFrom(
      this.http.post('/api/auth/login', payload, { withCredentials: true })
    );
  }

  private async encrypt(data: object): Promise<EncryptedPayload> {
    const { kid, publicKey, nonce } = await firstValueFrom(
      this.http.get<HandshakeResponse>('/api/auth/handshake', { withCredentials:
    );

    const rsaKey = await crypto.subtle.importKey(
      'spki', this.b64ToBuf(publicKey),
      { name: 'RSA-OAEP', hash: 'SHA-256' }, false, ['encrypt']
    );

    const aesKey = await crypto.subtle.generateKey(
      { name: 'AES-GCM', length: 256 }, true, ['encrypt']
    );
    const iv = crypto.getRandomValues(new Uint8Array(12));

    const plaintext = new TextEncoder().encode(JSON.stringify({ ...data, nonce }))
    const ct = await crypto.subtle.encrypt(
```

```
      { name: 'AES-GCM', iv, additionalData: new TextEncoder().encode(kid), tagLe
      aesKey, plaintext
    );

    const rawAes = await crypto.subtle.exportKey('raw', aesKey);
    const ek = await crypto.subtle.encrypt({ name: 'RSA-OAEP' }, rsaKey, rawAes);

    return { kid, ek: this.bufToB64(ek), iv: this.bufToB64(iv), ct: this.bufToB64
  }

  private b64ToBuf(b64: string): ArrayBuffer {
    const bin = atob(b64);
    const bytes = new Uint8Array(bin.length);
    for (let i = 0; i < bin.length; i++) bytes[i] = bin.charCodeAt(i);
    return bytes.buffer;
  }

  private bufToB64(buf: ArrayBuffer | Uint8Array): string {
    const bytes = buf instanceof Uint8Array ? buf : new Uint8Array(buf);
    let bin = '';
    for (let i = 0; i < bytes.length; i++) bin += String.fromCharCode(bytes[i]);
    return btoa(bin);
  }
}
```

Use it in your login component:

ts

```
await this.secureLogin.login(this.username, this.password, this.otp);
```

2. Spring Boot: `LoginCryptoService.java`

java

```java
package com.example.auth;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.web.server.ResponseStatusException;

import javax.crypto.Cipher;
import javax.crypto.spec.*;
import java.io.IOException;
import java.io.InputStream;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.security.*;
import java.security.spec.MGF1ParameterSpec;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

@Service
public class LoginCryptoService {

    public record EncryptedPayload(String kid, String ek, String iv, String ct) {

    private static final long NONCE_TTL_MS = 120_000;
    private static final SecureRandom RNG = new SecureRandom();

    private final PrivateKey privateKey;
    private final String publicKeyB64;
    private final String kid;
    private final ObjectMapper mapper;
    private final Map<String, Long> nonces = new ConcurrentHashMap<>(); // use Re

    public LoginCryptoService(
            @Value("${login.crypto.keystore}") String keystorePath,
            @Value("${login.crypto.keystore-password}") String keystorePassword,
            @Value("${login.crypto.alias}") String alias,
            ObjectMapper mapper) throws Exception {

        KeyStore ks = KeyStore.getInstance("PKCS12");
        try (InputStream in = Files.newInputStream(Path.of(keystorePath))) {
            ks.load(in, keystorePassword.toCharArray());
        }
        this.privateKey = (PrivateKey) ks.getKey(alias, keystorePassword.toCharAr
```

```
        this.publicKeyB64 = Base64.getEncoder()
                .encodeToString(ks.getCertificate(alias).getPublicKey().getEncode
        this.kid = alias;
        this.mapper = mapper;
    }

    public Map<String, String> handshake() {
        long now = System.currentTimeMillis();
        nonces.entrySet().removeIf(e -> e.getValue() < now);

        byte[] n = new byte[16];
        RNG.nextBytes(n);
        String nonce = Base64.getEncoder().encodeToString(n);
        nonces.put(nonce, now + NONCE_TTL_MS);

        return Map.of("kid", kid, "publicKey", publicKeyB64, "nonce", nonce);
    }

    public JsonNode decrypt(EncryptedPayload p) {
        byte[] aesRaw = null;
        try {
            if (p == null || !kid.equals(p.kid())) throw unauthorized();

            // Must be SHA-256 for BOTH digest and MGF1 to match Web Crypto
            Cipher rsa = Cipher.getInstance("RSA/ECB/OAEPPadding");
            rsa.init(Cipher.DECRYPT_MODE, privateKey, new OAEPParameterSpec(
                    "SHA-256", "MGF1", MGF1ParameterSpec.SHA256, PSource.PSpecifi
            aesRaw = rsa.doFinal(b64(p.ek()));

            Cipher gcm = Cipher.getInstance("AES/GCM/NoPadding");
            gcm.init(Cipher.DECRYPT_MODE, new SecretKeySpec(aesRaw, "AES"),
                    new GCMParameterSpec(128, b64(p.iv())));
            gcm.updateAAD(kid.getBytes(StandardCharsets.UTF_8));
            byte[] plain = gcm.doFinal(b64(p.ct()));

            JsonNode body = mapper.readTree(plain);
            Arrays.fill(plain, (byte) 0);

            String nonce = body.path("nonce").asText(null);
            Long expiry = (nonce == null) ? null : nonces.remove(nonce); // singl
            if (expiry == null || expiry < System.currentTimeMillis()) throw unau

            return body;
        } catch (GeneralSecurityException | IOException | IllegalArgumentExceptio
            throw unauthorized();
        } finally {
```

```
            if (aesRaw != null) Arrays.fill(aesRaw, (byte) 0);
        }
    }

    private static byte[] b64(String s) { return Base64.getDecoder().decode(s); }

    private static ResponseStatusException unauthorized() {
        return new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Invalid cred
    }
}
```

3. Spring Boot: `AuthController.java`

java

```java
package com.example.auth;

import com.fasterxml.jackson.databind.JsonNode;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.Map;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final LoginCryptoService crypto;

    public AuthController(LoginCryptoService crypto) {
        this.crypto = crypto;
    }

    @GetMapping("/handshake")
    public Map<String, String> handshake() {
        return crypto.handshake();
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginCryptoService.EncryptedPaylo
        JsonNode body = crypto.decrypt(payload);

        String username = body.path("username").asText();
        String password = body.path("password").asText();
        String otp      = body.path("otp").asText();

        // 1. passwordEncoder.matches(password, user.getPasswordHash())
        // 2. otpService.verifyAndConsume(user, otp)
        // 3. rate-limit / lockout
        // 4. set HttpOnly; Secure; SameSite session cookie

        return ResponseEntity.ok().build();
    }
}
```

4. Key generation (on the Spring Boot server)

```bash
```

```
keytool -genkeypair -alias login-2026-10 -keyalg RSA -keysize 3072 \
  -validity 365 -storetype PKCS12 -keystore /opt/app/keys/login-crypto.p12 \
  -dname "CN=login-crypto"
chmod 600 /opt/app/keys/login-crypto.p12
```

5. `application.properties`

properties

```
login.crypto.keystore=/opt/app/keys/login-crypto.p12
login.crypto.keystore-password=${LOGIN_KEYSTORE_PASSWORD}
login.crypto.alias=login-2026-10
```

Set `LOGIN_KEYSTORE_PASSWORD` as an environment variable on the Spring server, never in git. If Spring Security is enabled, permit `/api/auth/handshake` and `/api/auth/login` without authentication.

2 days ago

What if the payload is intercepted and new payload is created?

yesterday

A new payload is useless to an attacker unless they already know the victim's password and OTP. The encryption doesn't decide who gets in. The password and OTP checks behind it do. Here's how each attack plays out.

1. Tampering with the intercepted payload
The attacker can't decrypt it because they don't have the private key. If they change any byte of `ct`, `iv`, `ek` or `kid`, the AES-GCM authentication tag fails and the server returns 401. GCM provides integrity, not just secrecy.

2. Replaying it unchanged
The nonce inside was deleted on first use, so a replay gets a 401.

3. Building a brand-new payload
Anyone can call `/handshake`, get the public key and a nonce, and encrypt *something*. That's no different from typing into the login form, because the public key is meant to be public. But they still need the correct password and the current OTP, and they can't get those from the intercepted request since it's encrypted. If their guesses are wrong, they get a 401.

4. Scripting new payloads to brute-force
This is the real risk, and the controls that stop it are on the server, not in the encryption:

- **Lockout:** lock the account after about 5 failed password attempts.
- **OTP limits:** each OTP allows about 3–5 tries and expires in 2–5 minutes. Brute-forcing a 6-digit OTP this way is practically impossible.
- **Rate limits:** per IP and per user on both `/handshake` and `/login`. Nginx `limit_req` or Bucket4j works well.
- **CAPTCHA** after repeated failures.

5. Reading the plaintext on the victim's device
If an attacker can run Burp with a trusted CA on the victim's machine, or malware sees the data before encryption, no client-side scheme helps. That's endpoint compromise and outside what this design can protect against. Say so explicitly in the audit response.

Optional hardening: bind the nonce to the session
This stops a nonce issued to one browser from being used by another. Store the session ID with the nonce:

java

```java
// in handshake(HttpSession session)
nonces.put(nonce, new NonceEntry(session.getId(), now + NONCE_TTL_MS));

// in decrypt(..., HttpSession session)
NonceEntry e = nonces.remove(nonce);
if (e == null || e.expiry() < System.currentTimeMillis()
        || !e.sessionId().equals(session.getId())) throw unauthorized();
```

java

```java
record NonceEntry(String sessionId, long expiry) {}
```

Pass `HttpSession session` into both controller methods. Angular already sends `withCredentials: true`, so the session cookie goes with both requests.

For the auditors: confidentiality comes from RSA-OAEP plus AES-GCM, integrity from the GCM tag, anti-replay from the single-use, session-bound nonce, and authentication from the password, OTP, lockout and rate limiting.

yesterday

In the code how is the object mapper instantiated

15 minutes ago

You don't create it yourself. **Spring Boot creates the `ObjectMapper` and injects it** through the constructor.

When `spring-boot-starter-web` is on the classpath, Spring Boot's Jackson auto-configuration registers an `ObjectMapper` bean at startup. Because `LoginCryptoService` is a `@Service` with a single constructor, Spring sees the `ObjectMapper mapper` parameter and passes in that bean automatically:

java

```
public LoginCryptoService(
        @Value("${login.crypto.keystore}") String keystorePath,
        @Value("${login.crypto.keystore-password}") String keystorePassword,
        @Value("${login.crypto.alias}") String alias,
        ObjectMapper mapper) {    // <- injected by Spring
    ...
    this.mapper = mapper;
}
```

No `@Autowired` is needed for a single constructor, and no `new ObjectMapper()`.

One thing to check is your Spring Boot version.

- **Spring Boot 2.x or 3.x (Jackson 2):** the code works as written, with `import com.fasterxml.jackson.databind.ObjectMapper;`.
- **Spring Boot 4.x (Jackson 3 by default):** the package changes, and Jackson's exceptions become unchecked. Change the imports:

java

```
import tools.jackson.databind.JsonNode;
import tools.jackson.databind.ObjectMapper;
```

Then change the catch block, because `IOException` is no longer thrown and the compiler will reject it:

java

```
} catch (GeneralSecurityException | tools.jackson.core.JacksonException
         | IllegalArgumentException e) {
    throw unauthorized();
}
```

If the bean isn't found (for example, in a project without the web starter), define one yourself:

java

```
@Configuration
public class JacksonConfig {
    @Bean
    public ObjectMapper objectMapper() {
        return new ObjectMapper();
    }
}
```

Avoid `new ObjectMapper()` inside the service. The injected bean is thread-safe, shared, and picks up any Jackson settings in your `application.properties`.

15 minutes ago

I already have a externalUserAuthImp which is a service and i want to add this code in that class that class is being used in the controller by the means of autowired

13 minutes ago

Since your class uses `@Autowired` fields rather than constructor injection, load the key in a `@PostConstruct` method. A constructor runs before field injection, so the `@Value` and `@Autowired` fields would still be null there.

1. Interface (if you have one)

Your controller probably autowires the interface, so add the two methods there:

java

```java
public interface ExternalUserAuth {
    // ... your existing methods

    Map<String, String> handshake(HttpSession session);
    JsonNode decryptLoginPayload(EncryptedPayload payload, HttpSession session);
}
```

2. `EncryptedPayload.java` (separate file)

java

```java
package com.example.auth;

public record EncryptedPayload(String kid, String ek, String iv, String ct) {}
```

3. Add to `ExternalUserAuthImp`

java

```java
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import jakarta.annotation.PostConstruct;
import jakarta.servlet.http.HttpSession;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.web.server.ResponseStatusException;

import javax.crypto.Cipher;
import javax.crypto.spec.*;
import java.io.IOException;
import java.io.InputStream;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.security.*;
import java.security.spec.MGF1ParameterSpec;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;

@Service
public class ExternalUserAuthImp implements ExternalUserAuth {

    // ---- your existing fields stay as they are ----

    @Autowired
    private ObjectMapper mapper;

    @Value("${login.crypto.keystore}")
    private String keystorePath;

    @Value("${login.crypto.keystore-password}")
    private String keystorePassword;

    @Value("${login.crypto.alias}")
    private String kid;

    private static final long NONCE_TTL_MS = 120_000;
    private static final SecureRandom RNG = new SecureRandom();

    private PrivateKey privateKey;
    private String publicKeyB64;
    private final Map<String, NonceEntry> nonces = new ConcurrentHashMap<>();
```

```
    private record NonceEntry(String sessionId, long expiry) {}

    @PostConstruct
    void loadKeys() throws Exception {
        KeyStore ks = KeyStore.getInstance("PKCS12");
        try (InputStream in = Files.newInputStream(Path.of(keystorePath))) {
            ks.load(in, keystorePassword.toCharArray());
        }
        privateKey = (PrivateKey) ks.getKey(kid, keystorePassword.toCharArray());
        publicKeyB64 = Base64.getEncoder()
                .encodeToString(ks.getCertificate(kid).getPublicKey().getEncoded(
    }

    @Override
    public Map<String, String> handshake(HttpSession session) {
        long now = System.currentTimeMillis();
        nonces.entrySet().removeIf(e -> e.getValue().expiry() < now);

        byte[] n = new byte[16];
        RNG.nextBytes(n);
        String nonce = Base64.getEncoder().encodeToString(n);
        nonces.put(nonce, new NonceEntry(session.getId(), now + NONCE_TTL_MS));

        return Map.of("kid", kid, "publicKey", publicKeyB64, "nonce", nonce);
    }

    @Override
    public JsonNode decryptLoginPayload(EncryptedPayload p, HttpSession session)
        byte[] aesRaw = null;
        try {
            if (p == null || !kid.equals(p.kid())) throw unauthorized();

            Cipher rsa = Cipher.getInstance("RSA/ECB/OAEPPadding");
            rsa.init(Cipher.DECRYPT_MODE, privateKey, new OAEPParameterSpec(
                    "SHA-256", "MGF1", MGF1ParameterSpec.SHA256, PSource.PSpecifi
            aesRaw = rsa.doFinal(b64(p.ek()));

            Cipher gcm = Cipher.getInstance("AES/GCM/NoPadding");
            gcm.init(Cipher.DECRYPT_MODE, new SecretKeySpec(aesRaw, "AES"),
                    new GCMParameterSpec(128, b64(p.iv())));
            gcm.updateAAD(kid.getBytes(StandardCharsets.UTF_8));
            byte[] plain = gcm.doFinal(b64(p.ct()));

            JsonNode body = mapper.readTree(plain);
            Arrays.fill(plain, (byte) 0);
```

```
            String nonce = body.path("nonce").asText(null);
            NonceEntry e = (nonce == null) ? null : nonces.remove(nonce);
            if (e == null || e.expiry() < System.currentTimeMillis()
                    || !e.sessionId().equals(session.getId())) throw unauthorized

            return body;
        } catch (GeneralSecurityException | IOException | IllegalArgumentExceptio
            throw unauthorized();
        } finally {
            if (aesRaw != null) Arrays.fill(aesRaw, (byte) 0);
        }
    }

    private static byte[] b64(String s) { return Base64.getDecoder().decode(s); }

    private static ResponseStatusException unauthorized() {
        return new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Invalid cred
    }

    // ---- your existing methods stay as they are ----
}
```

4. Controller

java

```
@Autowired
private ExternalUserAuth externalUserAuth;

@GetMapping("/handshake")
public Map<String, String> handshake(HttpSession session) {
    return externalUserAuth.handshake(session);
}

@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody EncryptedPayload payload, HttpSession
    JsonNode body = externalUserAuth.decryptLoginPayload(payload, session);

    String username = body.path("username").asText();
    String password = body.path("password").asText();
    String otp      = body.path("otp").asText();

    // call your existing password + OTP verification here
    return ResponseEntity.ok().build();
}
```

Notes

- On Spring Boot 2.x, change `jakarta.*` imports to `javax.annotation.PostConstruct` and `javax.servlet.http.HttpSession`. On Boot 4.x, use the Jackson 3 imports and catch block from my last message.
- If your class has no interface and the controller autowires `ExternalUserAuthImp` directly, skip step 1 and remove the `@Override` annotations.
- If `@PostConstruct` fails, for example because of a wrong path or password, the app won't start. That's intentional: it should fail loudly rather than run without its key.

13 minutes ago